SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

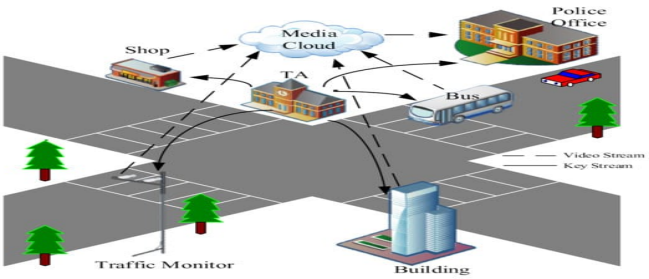
1. Smart City Surveillance System – CPU, Memory, I/O Interaction and Bus Structures

Introduction

A smart city surveillance system continuously captures and processes video streams from multiple cameras. The CPU, memory, and I/O devices must work together efficiently. During peak hours, large amounts of video data can create bottlenecks, leading to lag and delayed threat detection.

Interaction between CPU, Memory, and I/O

- ✓ **Input Devices (Cameras):** Continuously generate
- ✓ high-resolution video data.
- ✓ **Memory (RAM):** Temporarily stores video ✓ frames before processing.
- ✓ **CPU:** Fetches video data from memory. ✓ processing algorithms, and sends results to
- ✓ storage or monitoring systems.
- ✓ **Output Devices:** Display processed video or trigger alarms.
- ✓ During peak traffic:
- ✓ Cameras produce huge amounts of data.
- ✓ Memory bandwidth becomes overloaded.
- ✓ CPU waits for data from memory.
- ✓ I/O devices compete for bus access.



Role of Bus Structures

Single-Bus Structure

- One common bus connects CPU, memory, and I/O devices.
- Only one data transfer can occur at a time.
- Heavy traffic causes bus contention.
-

-

- CPU frequently waits for I/O operations.

Advantages

Simple design

Low cost

Disadvantages

- Lower throughput
- Increased waiting time
- Poor scalability for large surveillance systems

Multi-Bus Structure

- Separate buses for CPU, memory, and I/O.
- Multiple transfers occur simultaneously.
- Memory access and I/O operations can happen in parallel.

Advantages

- Higher bandwidth
- Reduced bus congestion
- Faster data transfer
- Better scalability

Disadvantages

- Higher hardware cost
- More complex design

Improving Instruction Execution Cycle

The instruction cycle consists of:

1. **Fetch** – Retrieve instruction from memory.
2. **Decode** – Interpret instruction.
3. **Execute** – Perform operation.
4. **Memory Access** – Read/write data.

-

•

5. **Write Back** – Store result.

Performance improvements include:

- **Instruction pipelining** allows multiple instructions to execute simultaneously.

  Cache memory reduces memory access time.

  DMA (Direct Memory Access) transfers camera data directly to memory without CPU involvement.
- **Branch prediction** minimizes delays during conditional instructions.
- **Multi-core processors** distribute video-processing tasks across several cores.

Overall Performance Enhancement

- Use a **multi-bus architecture** to reduce communication bottlenecks.
- Add **high-speed cache memory**.
- Implement **DMA** for faster I/O transfers.
- Use **pipeline execution** and **parallel processing**.
- Increase memory bandwidth.

Conclusion

The surveillance system's delay mainly results from excessive data transfer between CPU, memory, and I/O devices. Replacing a single-bus architecture with a multi-bus system and improving the instruction execution cycle through pipelining, cache memory, DMA, and multicore processing significantly enhances throughput and enables faster threat detection.

2. Real-Time Stock Trading Application – Fetch–Decode–Execute Cycle and CPI Reduction

Introduction

A stock trading application executes millions of instructions every second. During heavy market activity, delays occur because the processor experiences a high **Cycles Per Instruction (CPI)** due to frequent memory accesses and branch instructions.

•

•

Fetch–Decode–Execute Cycle

Every instruction passes through three major stages:

1. Fetch

CPU fetches the instruction from memory.

Program Counter (PC) is updated.

2. Decode

- CPU interprets the instruction.
- Determines required operands and operation. **3. Execute**
- Arithmetic or logical operation is performed.
- Memory is accessed if needed.
- Result is written back.

Impact on Execution Time

Execution Time is given by:

Execution Time = Instruction Count × CPI × Clock Cycle Time

A high CPI increases execution time.

Reasons for high CPI include:

- Frequent memory accesses
- Cache misses
- Branch instructions causing pipeline stalls
- Data hazards
- Instruction dependencies

Methods to Reduce CPI

1. Cache Memory

- Stores frequently used instructions and data.
- Reduces memory access delay.

•

-

- Lowers CPI.

---

2. Instruction Pipelining

- Multiple instructions execute simultaneously.

  Improves CPU utilization.

  Increases throughput.

---

3. Branch Prediction

- Predicts the outcome of branch instructions.
- Prevents pipeline flushing.
- Reduces branch penalties.

---

4. Out-of-Order Execution

- CPU executes independent instructions while waiting for memory.
- Improves processor efficiency.

---

5. Superscalar Architecture

- Executes multiple instructions in one clock cycle.
- Significantly lowers CPI.

---

6. Larger Cache and Prefetching • Fetches data before it is required.

- Reduces waiting time.

---

7. Multi-Core Processing

- Distributes trading tasks across multiple processor cores.
- Handles high transaction loads efficiently.

Architectural Enhancements

- High-speed L1, L2, and L3 cache
-

-

- Pipeline optimization
- Branch Target Buffer (BTB)
- Speculative execution

  Register renaming

-

•

High-bandwidth memory

- Multi-core processors

Conclusion

The fetch–decode–execute cycle directly affects execution time. Frequent memory access and branch instructions increase CPI and slow transaction processing. Architectural enhancements such as cache memory, pipelining, branch prediction, superscalar execution, and multicore processors reduce CPI and significantly improve trading system performance.

4. 8086 Microprocessor – Segmented Memory, Instruction Execution, and Addressing Modes (10 Marks)

Introduction

The **8086 microprocessor** uses a **segmented memory architecture** to access up to **1 MB of memory** using 16-bit registers. Memory is divided into different segments such as **Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES)**. Proper management of these segments is essential for correct execution of multiple programs. Incorrect handling can result in **wrong data access, stack overflow, and program execution errors**.

8086 Segmentation

The 8086 divides memory into four logical segments:

1. Code Segment (CS)

- Stores the program instructions.
- The **Code Segment (CS)** register points to the beginning of the code.
- The **Instruction Pointer (IP)** contains the offset of the next instruction.
- Physical Address = **CS × 10H + IP**

Function:

- Fetches instructions for execution.
- Ensures correct program flow.

---

2. Data Segment (DS)

- Stores variables, constants, and program data.
- The **Data Segment (DS)** register points to the data area.

Function:

- Used during data transfer and arithmetic operations.
- Accessed by instructions such as **MOV**, **ADD**, and **SUB**.

---

3. Stack Segment (SS)

- Stores temporary data, return addresses, and register values.
- Uses the **Stack Pointer (SP)** and **Base Pointer (BP)**.

Function:

- Supports function calls and interrupts.
- Implements Last-In-First-Out (LIFO) operations using **PUSH** and **POP** instructions.

---

4. Extra Segment (ES)

- Additional data segment mainly used in string operations.
- Works with the **Destination Index (DI)** register.

Function:

- Stores destination data during block transfer operations.

How Improper Segment Handling Causes Problems

1. Incorrect Data Access

If the **Data Segment (DS)** register points to the wrong memory location:

- Incorrect variables are read.
- Wrong data is modified.
- Program produces incorrect results.

Example:

- DS is not initialized correctly before executing a **MOV** instruction.
- The processor accesses an unintended memory location.

---

2. Stack Overflow

A stack overflow occurs when:

- Too many **PUSH** instructions are executed without corresponding **POP** operations.
- Recursive function calls exceed stack capacity.
- The **Stack Pointer (SP)** moves beyond the allocated stack segment.

Effects:

- Corrupted data
- Program crash
- Incorrect return addresses

---

3. Code Execution Errors

If the **Code Segment (CS)** or **Instruction Pointer (IP)** is modified incorrectly:

- Wrong instructions are fetched.
- Unexpected jumps occur.
- Program execution becomes unpredictable.

Instruction Execution Cycle in 8086

The instruction execution cycle consists of:

1. Fetch

- CPU fetches the instruction from the **Code Segment (CS)** using the **Instruction Pointer (IP)**.

2. Decode

- The instruction is interpreted by the control unit.
- Required operands and addressing mode are identified.

3. Execute

- The Arithmetic Logic Unit (ALU) performs the required operation.
- Data is read from or written to memory if necessary.

4. Write Back

- The result is stored in a register or memory location.

- The **IP** is updated to fetch the next instruction.

Efficient instruction execution ensures faster and error-free program operation.

Addressing Modes in 8086

1. Immediate Addressing

- Operand is included within the instruction.

Example:

MOV AX, 1234H

---

2. Register Addressing

- Operand is stored in a register.

Example:

MOV AX, BX

---

3. Direct Addressing

- Memory address is specified directly.

Example:

MOV AX, [2000H]

---

4. Register Indirect Addressing

- Address is stored in registers such as **BX**, **SI**, or **DI**.

Example:

MOV AX, [BX]

---

5. Based/Indexed Addressing

- Effective address is calculated using base and index registers.

Example:

MOV AX, [BX+SI]

Managing Segmentation and Addressing Modes

To ensure correct execution:

- Initialize **CS, DS, SS, and ES** properly before program execution.
- Allocate sufficient stack memory.
- Balance every **PUSH** instruction with a corresponding **POP**.
- Use the correct addressing mode for accessing variables.
- Keep the **Instruction Pointer (IP)** and segment registers updated correctly.
- Validate memory addresses before accessing data.
- Avoid overwriting stack or code segment memory.

Advantages of Proper Segmentation

- Efficient memory organization
- Supports execution of multiple programs
- Faster memory access
- Better program modularity
- Reduced memory conflicts
- Improved reliability and system stability

Conclusion

The **8086 segmented memory architecture** organizes memory into **Code, Data, Stack, and Extra Segments**, enabling efficient use of 1 MB memory. Incorrect handling of segment registers can lead to **wrong data access, stack overflow, and execution errors**. Proper initialization of segment registers, careful management of the instruction execution cycle, correct use of addressing modes, and balanced stack operations ensure accurate, reliable, and efficient program execution in multitasking systems.